



本科生实验报告

实验课程：并行设计与算法

实验名称：自动生成词法分析程序

专业名称：计算机科学与技术

实验地点：东校园实验中心大楼 B202

报告时间：2026/5/31

学号	23320143
姓名	唐雅妍
邮箱	tangyy77@mail2.sysu.edu.cn

目录

1	总结 Oberon-0 语言的词汇表	2
2	抽取 Oberon-0 语言的词法规则	2
2.1	正则定义	2
2.2	词法规则异同	3
3	生成 Oberon-0 语言的词法分析程序	4
4	实验结果	5
5	不同词法工具的差异	6

1 总结 Oberon-0 语言的词汇表

通过对单词分类，能够更好的编写词法程序。在 Oberon-0 中关键字实际上是预定义标识符，而保留字为语法结构组成部分，因此需要区分。在实际处理时，保留字每个应当单独分配一个 Symbol 的标号，而关键字在词法分析时可以识别为标识符，并在语义处理中查表进行对应处理，如语义处理中 INTEGER 标记为 TYPE 的一个标识符，而 READ 标记为 PROCEDURE 的一个标识符。

按找单词的作用来对单词进行分类，需要注意的是部分单词存在多种语义，如 = 存在两种作用，如关系运算符和定义标识符类型等，因此会出现在两个表中。除此之外，Oberon-0 的运算符并不全是符号，也包括字母组成的保留字形式，如 DIV 等。

表 1: 保留字与关键字

保留字	关键字
MODULE, PROCEDURE, BEGIN, END, CONST, TYPE, VAR, ARRAY, OF, RECORD, WHILE, DO, IF, THEN, ELIF, ELSE	INTEGER, BOOLEAN, read, write, writeln

表 2: 运算符表

类别	符号
赋值运算符	:=
关系运算符	= # < <= > >=
一元算术运算符	+ -
二元算术运算符	+ - * DIV MOD
逻辑运算符	& OR ~

表 3: 其他词法符号

类别	内容
分隔符/选择符	; . () , []
定义符	= :
注释	(* *)
标识符	letter(letter digit)*
整数数字	digit+

2 抽取 Oberon-0 语言的词法规则

2.1 正则定义

这里由于注释不支持嵌套，因此判断 (* 匹配到第一个 *) 退出，中间不能有其他 *)。

$$\begin{aligned}\text{digit} &\rightarrow [0-9] \\ \text{integer} &\rightarrow \text{digit}^+ \\ \text{identifier} &\rightarrow \text{letter} (\text{letter} \mid \text{digit})^* \\ \text{comment} &\rightarrow "(*" ([*] \mid "*"^+ [])^* ")"\end{aligned}$$

由于 Oberon-0 支持八进制输入，因此可以细分为：

$$\begin{aligned}\text{DecInteger} &\rightarrow [1-9] (\text{digit})^* \\ \text{OctInteger} &\rightarrow 0 ([0-7])^*\end{aligned}$$

2.2 词法规则异同

Oberon-0 的词法规则相较于 C/C++ 和 Java 更为简洁，其设计目标主要是服务于教学和编译原理实验，因此省略了许多工程化语言中的复杂特性。

在数字常量方面，Oberon-0 仅支持十进制整数，不支持浮点数、科学计数法以及二进制、十六进制等多种数字表示形式；而 C/C++ 和 Java 则支持更加丰富的数值类型和字面量表示方式。

在注释方面，Oberon-0 仅支持形如 `(* ... *)` 的非嵌套注释。相比之下，Java 支持单行注释 `//`、多行注释 `/ ... /` 以及文档注释 `/* ... */` 等多种注释形式，具有更好的代码文档化能力。

在大小写处理方面，Oberon-0 不区分字母大小写，即关键字和标识符在识别时忽略大小写差异；而 C/C++ 和 Java 均区分大小写，因此 `var`、`Var` 和 `VAR` 会被视为不同的标识符。

在标识符定义方面，Oberon-0 要求标识符必须以字母开头，后续字符由字母和数字组成，并且通常会对标识符长度进行限制。而 C/C++ 和 Java 允许标识符以字母或下划线开头，字符组成也更加灵活。

此外，Oberon-0 不支持字符串常量，因此词法分析器无需处理字符串中的转义字符、多行字符串等复杂情况；而字符串是 C/C++ 和 Java 中的重要词法单元之一。

尽管存在上述差异，Oberon-0 与 C/C++、Java 在词法分析的基本思想上仍具有一致性。例如，三者都会在词法分析阶段忽略空格、制表符和换行符等空白字符，并通过有限自动机识别关键字、标识符、常量和运算符等词法单元。

3 生成 Oberon-0 语言的词法分析程序

根据官方文档，下载 jflex，官方提供了.jar 包，因此可以通过 -jar 来进行生成。

flex 文件的结构如 1 所示。

Listing 1: flex 文件结构

```
1      UserCode
2      %%
3      Options and declarations
4      %%
5      Lexical rules
```

UserCode 主要为一些初始化和头文件导入，在这里导入异常的包。

Options and declarations 声明、正则表达式定义和状态定义。声明定义如下：

- **%public**: 将生成的扫描器类声明为 **public**，使其能够被其他包或类访问。如果省略该选项，则生成的类采用默认访问权限。
- **%class OberonScanner**: 指定生成的词法分析器类名为 **OberonScanner**。JFlex 默认生成的类名为 **Yylex**，通过该指令可以自定义类名。
- **%yylexthrow LexicalException**: 声明词法分析函数 **yylex()** 可能抛出的异常类型。在本实验中，当检测到非法字符、未闭合注释等词法错误时，可以抛出 **LexicalException** 异常。
- **%unicode**: 启用 Unicode 字符集支持，因此支持中文注释。
- **%ignorecase**: 忽略大小写差异。
- **%line**: 启用行号统计功能。JFlex 将自动维护变量 **yyline**，记录当前扫描位置所在的行号（从 0 开始计数）。
- **%column**: 启用列号统计功能。JFlex 将自动维护变量 **yycolumn**，记录当前扫描位置所在的列号（从 0 开始计数）。
- **%debug**: 调试选项（本实验中未启用）。启用后，JFlex 会在生成的扫描器中添加调试输出代码，用于观察状态转移和匹配过程，便于词法分析器的调试。
- **%type String**: 指定 **yylex()** 函数的返回类型为 **String**，因此每条词法规则执行后返回一个字符串。

- `%{...%}`: 用户代码区。该区域中的 Java 代码会被原样复制到生成的扫描器类中。本实验在此定义了辅助函数 `debuginfo()`, 用于生成包含单词类型、行号、列号及匹配文本的调试信息。
- `%state COMMENT`: 定义一个名为 `COMMENT` 的词法状态。扫描器可以通过 `yybegin(XXX)` 进入该状态, 并在不同状态下使用不同的词法规则。本实验利用该状态实现 Oberon-0 的注释处理。

还需要处理不同环境下的换行和空白符号的处理, 对于空白符号, 跳过即可。

Lexical rules `jflex` 会遵循最长匹配原则进行处理, 相同长度的则选择第一个出现的表达式, 不过在此设计中, 不会出现相同长度的情况。因此, 可以通过 `[^]` 或者 `.` 来对没有匹配的字符进行报错, 但是要注意的是第二种方法不会匹配换行。

为了对注释能进行精准报错, 使用新增状态进行处理, 当遇到 `(*` 时会进入该状态, 如果在该状态下遇到文件末尾 (`<<EOF>>`) 则报错, 如果遇到 `*)` 则跳回初始状态 (`YYINITIAL`)。

需要特别注意数字和标识符之间要有空格隔开, 否则需要报错, 因此这里使用 `{Digit}+ {Identifier}` 匹配错误数字常量, 并抛出 `IllegalIntegerException` 异常。

`jflex` 提供了 `yylength()` 来获取匹配字符的长度, 他的速度会比对 `yytext()` 再计算长度更快, 可以用它来对标识符或数字常量长度不满足要求时抛出异常。

八进制数的报错同样与数字匹配报错的思路一致, 先匹配错误八进制 `0{Digit}*`, 再判断里面是否出现了 9 或 8。JAVA 的 `Integer.parseInt` 提供了对不同进制数的转换。特别的, 这里 0 会被识别为八进制数, 不过结果还是一样的。

4 实验结果

编译源程序生成的字节码, 并仿照 debug 模式下的 Main 函数形式写了一个测试代码, 返回识别的 Token, 将实验一的正确例子程序作为输入, 结果如图 1 所示。

将实验 1 的六个异常程序作为输入, 实现一个测试函数判断异常抛出的类型是否正确, 测试结果如图 2a 所示。

```
Identifier: at line 65, column 20: y  
COMMA  
Identifier: at line 65, column 23: ans  
RPAREN  
SEMICOLON  
KEYWORD  
LPAREN  
Identifier: at line 66, column 19: ans  
RPAREN  
RESERVED WORD  
RESERVED WORD  
Identifier: at line 68, column 9: Main  
RESERVED WORD  
Identifier: at line 69, column 5: Sample  
DOT  
EOF
```

图 1: 运行结果

5 不同词法工具的差异

GNU Flex、JLex 和 JFlex 均采用正则表达式和动作代码的方式定义词法规则，并遵循最长匹配原则。JFlex 是在 JLex 的基础上进行扩展。

- 实现语言不同，对于 JFlex 和 JLex 都是由 Java 实现并且生成 Java 扫描器的；而 GNU Flex 是由 C++ 实现，并且生成 C++ 扫描器。
- GNU Flex 默认面向 ASCII，因此处理中文比较困难；而 JLex 支持 Unicode 有限；JFlex 提供对 %unicode 的声明选项，完全支持 Unicode。
- 正则表达式、状态切换等具体语法会有些差异。

```
(base) PS E:\github-res\tyylab3\ex2> test.bat
===== Oberon Scanner Exception Test =====

Testing: ../testcases/gcd001.obr
Expected Exception: IllegalSymbolException

TEST PASSED
-----

Testing: ../testcases/gcd002.obr
Expected Exception: IllegalIntegerException

TEST PASSED
-----

Testing: ../testcases/gcd003.obr
Expected Exception: IllegalIntegerRangeException

TEST PASSED
-----

Testing: ../testcases/gcd004.obr
Expected Exception: IllegalOctalException

TEST PASSED
-----

Testing: ../testcases/gcd005.obr
Expected Exception: IllegalIdentifierLengthException

TEST PASSED
-----

Testing: ../testcases/gcd006.obr
Expected Exception: MismatchedCommentException

TEST PASSED
-----

===== Test Summary =====
Total: 6
Passed: 6
Failed: 0
=====
```

(a) 测试结果

```
RESERVED WORD
Identifier: at line 11, column 9: a
Unexpected exception:
exceptions.IllegalSymbolException: at line 11, column 10
    at OberonScanner.yylex(OberonScanner.java:781)
    at TestScanner.main(TestScanner.java:30)
```

(b) 非法字符

```
Identifier: at line 30, column 9: i
:=
Unexpected exception:
exceptions.IllegalIntegerException: Illegal Integer
    at OberonScanner.yylex(OberonScanner.java:911)
    at TestScanner.main(TestScanner.java:30)
```

(c) 非法整形数字

```
Identifier: at line 55, column 9: BinSearch
SEMICOLON
Unexpected exception:
exceptions.MismatchedCommentException: at line 69, column 12
    at OberonScanner.yylex(OberonScanner.java:768)
    at TestScanner.main(TestScanner.java:29)
```

(d) 注释匹配异常